

MGR840 – Téléphonie IP et Mobilité

Laboratoire #1

Numérisation de la voix et dimensionnement de réseau téléphonique

Date de remise : dimanche le 27 septembre à 23:59 sur Moodle

Fichiers de données fournis :

1. vad_dataset_train.csv : Jeu d'entraînement (10 000 trames audio pre-extraites).
2. vad_dataset_test.csv : Jeu de test (3 000 trames audio pre-extraites).
3. traffic_24h.csv : Relevé de trafic réseau (96 mesures sur 24 heures).

Remise:

1. Notebook Jupyter (MGR840_TP_Nom1_Nom2.ipynb) entièrement exécuté et commenté.
2. Rapport d'analyse au format PDF (MGR840_TP_Nom1_Nom2.pdf) de 4 pages maximum.

1. Survol, objectifs, et attentes

Ce laboratoire consiste à appliquer les concepts théoriques acquis dans le cours à des exemples pratiques afin d'assimiler les notions élémentaires de la numérisation des signaux vocaux et du dimensionnement de réseau téléphonique. Les objectifs pédagogiques sont : i) Maîtriser le chaîne de codage PCM de l'échantillonnage, la quantification non-uniforme et le et multiplexage TDM, ii) Appliquer l'IA sur le traitement du signal vocal afin de Concevoir un classifieur VAD (*Voice Activity Detection*) pour réduire le débit utile par voie, iii) Modéliser les files d'attente et le dimensionnement du réseau téléphonique en automatisant les calculs d'Erlang B/C et évaluer l'impact du trafic, et iv) Anticiper la charge de trafic à l'aide de séries temporelles.

2. Description du laboratoire

Ce laboratoire consiste en deux volets : la numérisation de la voix et le dimensionnement du réseau téléphonique.

2.1. Numérisation PCM, Quantification Loi- μ et Analyse Voyelles/Consonnes

La chaîne de numérisation vocalique normalisée ITU-T G.711 repose sur un échantillonnage PAM (*Pulse Amplitude Modulation*) à $f_s = 8$ kHz ($T_s = 125$ μ s). Au Canada, la numérisation applique une quantification non-uniforme sur 8 bits (256 niveaux) selon la loi- μ ($\mu = 255$).

La voix humaine présente une dynamique d'amplitude très étendue : i) Voyelles (sons voisés) avec des caractéristiques comme forte énergie, signaux quasi-périodiques, amplitudes élevées ($|x| \in [0,4; 1,0]$ V), et ii) Consonnes (fricatives) qui ont de faible énergie, de nature aléatoire/bruitée, et de faibles amplitudes ($|x| \in [0,01; 0,08]$ V).

En quantification uniforme 8 bits, le pas q est constant sur toute l'échelle $[-V_{\max}, +V_{\max}]$:

$$q = \frac{2V_{\max}}{2^N} = \frac{2}{256} = 0,0078125 \text{ V}$$

Le bruit de quantification moyen $P_e = \frac{q^2}{12}$ étant fixe, le rapport signal-sur-bruit de quantification ($SQNR$) s'effondre pour les signaux:

$$SQNR_{\text{dB}} = 10 \log_{10} \left(\frac{P_{\text{signal}}}{P_e} \right)$$

La loi- μ applique une compression logarithmique avant quantification uniforme:

$$y = \text{sgn}(x) \cdot \frac{\ln(1 + \mu|x|)}{\ln(1 + \mu)}, \quad \text{avec } \mu = 255 \text{ et } |x| \leq 1$$

Cette transformation garantit une erreur relative $\frac{\Delta x}{|x|}$ approximativement constante, maintenant un $SQNR \approx 38 \text{ dB}$ uniforme sur une plage dynamique de plus de 40 dB.

2.1.1. Tâches à réaliser

1. Générer deux signaux synthétiques d'une durée de 100 ms à $f_s = 8 \text{ kHz}$:
 - Voyelle (/a/) : Sinusoïde composite $f = 800 \text{ Hz}$ d'amplitude $A = 0,8 \text{ V}$.
 - Consonne (/s/) : Bruit blanc filtré passe-haut d'amplitude peak $A = 0,04 \text{ V}$.
2. Implémenter la quantification uniforme 8 bits et la compression/décompression Loi- μ .
3. Calculer le $SQNR$ en dB pour les deux phonèmes sous les deux régimes de quantification.
4. Comparer le $SQNR$ du son de voyelle et de consonne.

Un fichier de code source (partiel.py), disponible sur le site Moodle du cours, contient des fonctions de base pour réaliser ces tâches ci-dessus. Il vous faut remplir ce fichier et écrire votre rapport.

2.2. Classification VAD par l'apprentissage automatique

La détection d'activité vocale (Voice Activity Detection - VAD) est une brique critique des réseaux mobiles 5G et de la téléphonie IP (VoIP/VoNR). Elle agit comme un classifieur binaire traitant le flux trame par trame (10 ms):

$$y \in \{0 \text{ (Silence/Bruit ambiant)}, 1 \text{ (Voix Active)}\}$$

Afin de s'affranchir de la sensibilité aux variations de bruit des seuils statiques, on utilise un modèle d'apprentissage supervisé. Pour ce TP, deux jeux de données de descripteurs audio préextraits vous sont fournis sur le site Moodle du cours: vad_dataset_train.csv et vad_dataset_test.csv.

Variable	Type	Description
energy	Flottant	Énergie à court terme de la trame de 10 ms
zcr	Flottant	Taux de passage par zéro (<i>Zero-Crossing Rate</i>)
spectral_flatness	Flottant	Mesure de la platitude spectrale (distinction ton / bruit)
spectral_centroid	Flottant	Centre de gravité du spectre de fréquence (en Hz)
label	Entier	Cible (Target) : 0 = Silence, 1 = Voix active

2.2.1. Tâches à réaliser

1. Charger les fichiers vad_dataset_train.csv et vad_dataset_test.csv avec pandas.
2. Inspecter la distribution des classes (label) dans le jeu d'entraînement pour vérifier l'équilibre des données.
3. Normaliser les descripteurs (StandardScaler de scikit-learn).
4. Entraîner et comparer deux algorithmes de classification :
 - Modèle A : Régression Logistique (LogisticRegression)
 - Modèle B : Forêt Aléatoire (RandomForestClassifier)
5. Évaluer les modèles sur le jeu de test : afficher la matrice de confusion, la précision (*Accuracy*), le rappel (*Recall*) et le score F1.
6. Identifier le meilleur modèle pour une intégration réseau en temps réel.

Un fichier de code source (partie2.py), disponible sur le site Moodle du cours, contient des fonctions de base pour réaliser ces tâches ci-dessus. Il vous faut remplir ce fichier et écrire votre rapport.

2.3. Bilan d'économie de bande passante réseau

Une voie téléphonique numérique DS0 génère un débit constant :

$$R_{DS0} = 8000 \text{ échantillons/s} \times 8 \text{ bits/échantillon} = 64 \text{ kbit/s}$$

Sur un lien téléphonique, le silence occupe en moyenne 40% à 50% de la durée totale d'un appel. Lorsque le VAD détecte un silence ($y = 0$), le transcodeur interrompt l'émission de paquets (remplacés au récepteur par un générateur de bruit de confort local CNG).

Le facteur d'activité vocale α sur l'ensemble du jeu de test est défini par :

$$\alpha = \frac{\sum_{i=1}^{N_{\text{test}}} \hat{y}_i}{N_{\text{test}}}$$

Le débit effectif moyen par canal devient $R_{\text{eff}} = \alpha \times 64 \text{ kbit/s}$, libérant une capacité de transmission importante sur un lien agrégé ($T1 = 24 \text{ canaux DS0} = 1,544 \text{ Mbit/s}$).

2.3.1. Tâches à réaliser

1. À partir des prédictions $\hat{y}$ du meilleur modèle de la Partie 2 sur le jeu de test, calculer le facteur d'activité vocale α .
2. Dédurre le débit effectif moyen R_{eff} d'un canal DS0.
3. Calculer le pourcentage de bande passante économisé.
4. Évaluer le nombre théorique de communications simultanées qu'une liaison T1 (1,544 Mbit/s) peut supporter grâce au gain de la détection de silence VAD.

Un fichier de code source (partie3.py), disponible sur le site Moodle du cours, contient des fonctions de base pour réaliser ces tâches ci-dessus. Il vous faut remplir ce fichier et écrire votre rapport.

2.4. Dimensionnement Erlang B

La charge de trafic offerte A s'exprime en Erlangs ($A = \lambda/\mu$). Le modèle Erlang B calcule la probabilité de blocage P_b pour S canaux comme suit:

$$P_b(A, S) = \frac{\frac{A^S}{S!}}{\sum_{k=0}^S \frac{A^k}{k!}}$$

Pour éviter les erreurs de dépassement mémoire (*overflow*) liées aux factorielles sur de grands réseaux, le calcul s'effectue via la formule récursive de Fortet :

$$P_b(A, k) = \frac{A \cdot P_b(A, k-1)}{k + A \cdot P_b(A, k-1)} \quad \text{avec } P_b(A, 0) = 1$$

Un dimensionnement statique dimensionne le réseau pour l'Heure de Pointe (DPP), entraînant un surdimensionnement important durant les heures creuses. Dans l'approche de l'IA, on utilise un modèle prédictif XGBoost entraîné sur l'historique du trafic pour estimer la charge future $A(t)$ et ajuster dynamiquement la capacité $S(t) = \lceil n_{T1} \times 24 \rceil$ par pas de 15 minutes, sous contrainte SLA $P_b \leq 1\%$.

Le fichier de données traffic_24h.csv regroupe 96 mesures temporelles (24 heures découpées par pas de 15 minutes) enrichies de descripteurs d'historiques de charge précalculés :

Champ	Type	Description
timestamp	Chaîne	Horodatage de l'intervalle au format HH:MM (de 00:00 à 23:45).
hour	Entier	Heure de la journée (0 à 23).
minute	Entier	Minute de début de l'intervalle (0, 15, 30, 45).
is_peak_hour	Binaire	Flag indiquant la plage de haute fréquentation (1 entre 13h et 17h, 0 sinon).
traffic_lag_1	Flottant	Charge offerte observée à $t - 1$ (15 min plus tôt) en Erlangs.
traffic_lag_2	Flottant	Charge offerte observée à $t - 2$ (30 min plus tôt) en Erlangs.
traffic_lag_3	Flottant	Charge offerte observée à $t - 3$ (45 min plus tôt) en Erlangs.
traffic_lag_4	Flottant	Charge offerte observée à $t - 4$ (60 min plus tôt) en Erlangs.
offered_traffic_erlang	Flottant	Cible (Target) : Charge réelle observée à l'instant t en Erlangs.

2.4.1. Tâches à réaliser

1. Implémenter la fonction récursive de calcul d'Erlang B : $\text{erlang_b}(A, S)$.
2. Déterminer le nombre de canaux S_{stat} et le nombre de liaisons T1 fixes à déployer pour un trafic statique d'heure de pointe $A = 35$ Erlangs garantissant $P_b \leq 1\%$.
3. Importer le fichier traffic_24h.csv avec pandas.
4. Entraîner un modèle de régression XGBRegressor sur les 70 premiers pourcents des données chronologiques, puis prédire la charge sur les 30% restants (jeu de test).

5. Développer un algorithme d'Auto-Scaling qui, pour chaque prédiction de charge $\hat{A}(t)$, calcule le nombre minimal de canaux $S_{\text{dyn}}(t)$ requis pour maintenir $P_b \leq 1\%$.
6. Comparer la consommation totale de ressources (mesurée en canaux $\times$ heures) de l'Auto-Scaling IA avec celle du dimensionnement statique fixe d'heure de pointe sur la période de test.

Un fichier de code source (partie4.py), disponible sur le site Moodle du cours, contient des fonctions de base pour réaliser ces tâches ci-dessus. Il vous faut remplir ce fichier et écrire votre rapport.